The Journal of Open Source Software

PyTLidar: A Python Package for Tree QSM Modeling from Terrestrial Lidar Data

John Hagood [1¶], **Fan Yang** [1¶], **Shruti Motiwale** [3], **Breanna Shi** [1], **Jeffery B. Cannon** [2¶], and **James T. Stroud** [4¶]

1 College of Computing, Georgia Institute of Technology, United States **2** The Jones Center at Ichauway, United States **3** Pasteur Labs, Inc. **4** School of Biological Sciences, Georgia Institute of Technology, United States ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 29 August 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

PyTLidar is an open-source Python package that reconstructs 3D tree Quantitative Structure Models (QSM) from Terrestrial Lidar Scanning (TLS) data, providing a user-friendly tool that improves and expands upon the MATLAB-based TreeQSM method (Raumonen et al., 2013). QSMs are used to automate detection and calculation of various topological and volumetric measurements that would normally take great effort to gather in the field. PyTLidar provides an accessible, extensible, and GUI-driven workflow for researchers and practitioners in forestry, ecology, and 3D vegetation modeling to create QSMs. The package also integrates interactive visualization tools for inspecting model quality and viewing calculated tree properties. The ease of use and installation of PyTLidar provides ecologists an option to gather forest measurements that does not rely on proprietary tools.

The key features of PyTLidar are a reproduction of the TreeQSM core functionality, and enhancing the experience of setting up experiments and viewing results. It provides functionality for loading and extracting point cloud data from .las and .laz files as well as automatic calculation of a range of initial parameters for the QSM model based on point cloud structure. The QSM creation methods include generation of a Voronoi partition of the point cloud, segment detection, detection of parent-child relationships of branches, and cylinder fitting. PyTLidar also calculates various tree metrics such as branch length and volume and provides these results in text format as well as visual graphics. QSMs are also output in an interactive format which can be viewed directly in the GUI as well as saved and shared as an html file. All of this is packaged within a user-friendly GUI while also providing support for command line and direct Python interfacing.

Statement of Need

Terrestrial Laser Scanning (TLS) is an active remote sensing technology which uses infrared laser pulses to collect millions of three-dimensional coordinate points on the surface of objects, preserving spatial information and providing unprecedented detail on structural information. The technology is rapidly being adopted for diverse uses in forestry and ecology, as it is useful for estimating forest structure (Donager et al., 2021), aboveground biomass (AGB) (Atkins et al., 2025), canopy gap fraction and forest fuels (Loudermilk et al., 2023), crown shape (Zhu et al., 2020), disturbance patterns (Cannon et al., 2024), tree competition (Metz et al., 2013), physiology (Hakala et al., 2015), and other ecological properties. To realize the potential of TLS for use in forestry and ecological applications, accurate and efficient reconstruction of QSMs from TLS point cloud data is essential (Hackenberg et al., 2015).

The use of QSM software on point cloud data permits estimation of detailed components

of branch architecture such as branch diameter, volume, and stem taper (Lau et al., 2018), providing detailed information for fine-scale estimates of AGB, canopy architecture, and more. TreeQSM is a software that has been widely used in forestry and ecology for modeling tree structures from TLS point clouds (Terryn et al., 2020). Comparing to other similar softwares, TreeQSM stands out for speed, reliability, and ease of use, while SimpleForest (Jan et al., 2021) (available within Computree) seems to be similarly capable to TreeQSM, but is only available through Computree, which has been undergoing an extended upgrade process and lacks up-to-date documentation. AdQSM (Fan G, 2020) is extremely fast and simple but lacks many of the statistics and visualizations other tools have and has not been officially released by the authors. aRchi (Martin-Ducup M., 2022) provides various functions but takes significantly longer to create a QSM than comparable packages. 3dForest (Bubnik V., 2025) has a promising GUI but was unstable during testing, crashing when loading data. There is also a lack of viable options within Python specifically. While TreeQSM is used in many applications, its reliance on MATLAB makes it less accessible for users, and its lack of graphical interface makes the tool less user-friendly and its parameter tuning less efficient. Thus we aimed to port and improve TreeQSM.

PyTLidar addresses these issues by providing a native Python implementation of TreeQSM's core algorithms, wrapped in a streamlined graphical user interface that allows researchers to visualize and evaluate models. It promotes reproducible and exploratory research by offering transparent parameter control, open-source licensing, and seamless integration into Python-based analysis workflows. This work lowers the barrier for adoption of QSM modeling by removing the MATLAB dependency, enhancing accessibility for the broader open-source geospatial and ecological modeling community. PyTLidar is currently being used for ongoing projects in ecological monitoring and evolutionary field research.

Method

TreeQSM models individual trees from terrestrial lidar scans by first creating a Voronoi partition of the point cloud. This assigns the points within the cloud to an initial cluster based on its proximity to an initial point. The size of these regions, referred to as cover sets, are determined by the input patch diameters. Since the cover sets form the building blocks for reconstructing the tree's global shape, the selection of the patch diameter can have a major impact on the quality of the resulting QSM. Larger patch diameters will connect points further away, counteracting occlusion, with a corresponding loss in detail. Conversely, smaller patch diameters will increase detail, but be more susceptible to occlusion. As part of the construction of the initial partition, the algorithm also determines the neighboring cover sets. Based on topological investigation of neighboring cover sets, the point cloud is segmented into individual branches, with parent-children relationships of branches identified. This process is repeated, re-creating the cover sets using a range of patch diameters based on location along the tree. Points determined to be along the trunk use size values closer to the max patch diameter, while points further up the tree use values closer to min patch diameter. After the second segment detection, each branch is approximated as a collection of connected cylinders of varying radius, length, and orientation. The cylinders are fit using standard root-finding methods to minimize distance between points and approximated cylinder surface(Raumonen et al., 2013). This cylinder-based representation offers a simple yet effective regularization of the complex tree structure, supporting downstream analyses such as stem volume estimation or structural trait extraction (Markku et al., 2015).

Software Description

PyTLidar is organized into several key modules: core QSM algorithms (treeqsm.py), batch processing utilities (treeqsm_batch.py), GUI components built with PyQt6 (Python bindings for the Qt 6 framework), and visualization tools using Plotly. The software follows a modular

design that allows researchers to either use the complete GUI application or integrate individual components into their own Python workflows.

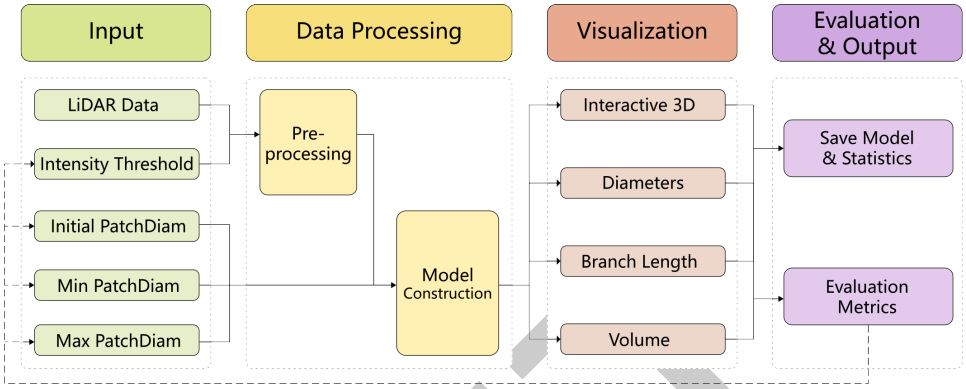

Figure 1: PyTLidar creates a QSM from an input point cloud and a set of parameters representing the size of the initial building blocks of the model. Structural measurements derived from the model and the model itself can then be viewed and evaluated within the tool.

When using the GUI, users can input or automatically generate values for key modeling parameters, including the minimum, and maximum patch diameters within a user-defined parameter (Figure 2). Also, an intensity threshold can be set to filter the point cloud data, helping to remove lidar returns due to noise or vegetation prior to modeling. Users may choose between batch processing of an entire directory of point cloud files or processing a single file. The GUI also includes options for displaying only the optimal model, based on performance metrics such as mean point distance to constructed surface.

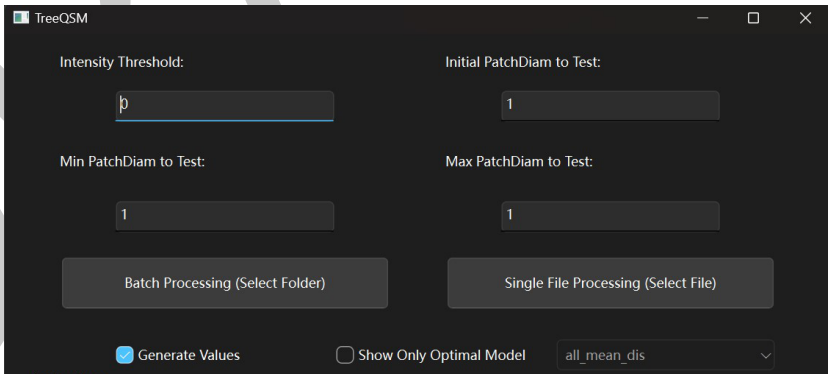

Figure 2: Graphical user interface for parameter input and data selection for PyTLidar.

After parameter and file selection, the software opens a new interface displaying data processing progress. Once the QSM reconstruction process is complete, PyTLidar provides interactive 3D visualization of the generated QSM using plotly (Figure 3). Users can inspect the structural fidelity of the reconstructed model, including trunk and branch geometry, and compare different parameter configurations for best fit. This combination of visual feedback and customizable processing offers an efficient path toward accurate and transparent tree structure analysis. If running in batch mode, users may also set the number of parallel cores to utilize to run simultaneous processes.

DRAFT

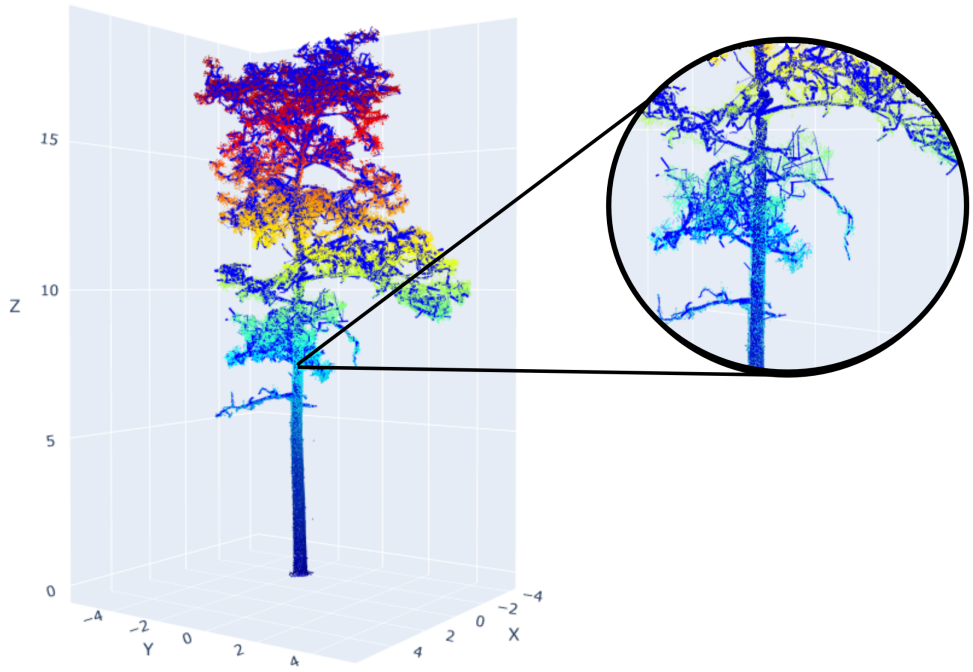

Figure 3: Point cloud and fitted cylinder of a sample pine. The graphical interface allows a user to compare the produced model (blue lines) directly to the point cloud (points colored by height) to visually evaluate fit.

Users can also review the morphological summaries of the QSM, including distribution of branch diameters, branch volume, surface area, and length with regard to diameter or order from stem, as with the original TreeQSM implementation (Figure 4). All of the produced figures are saved for later viewing and reference.

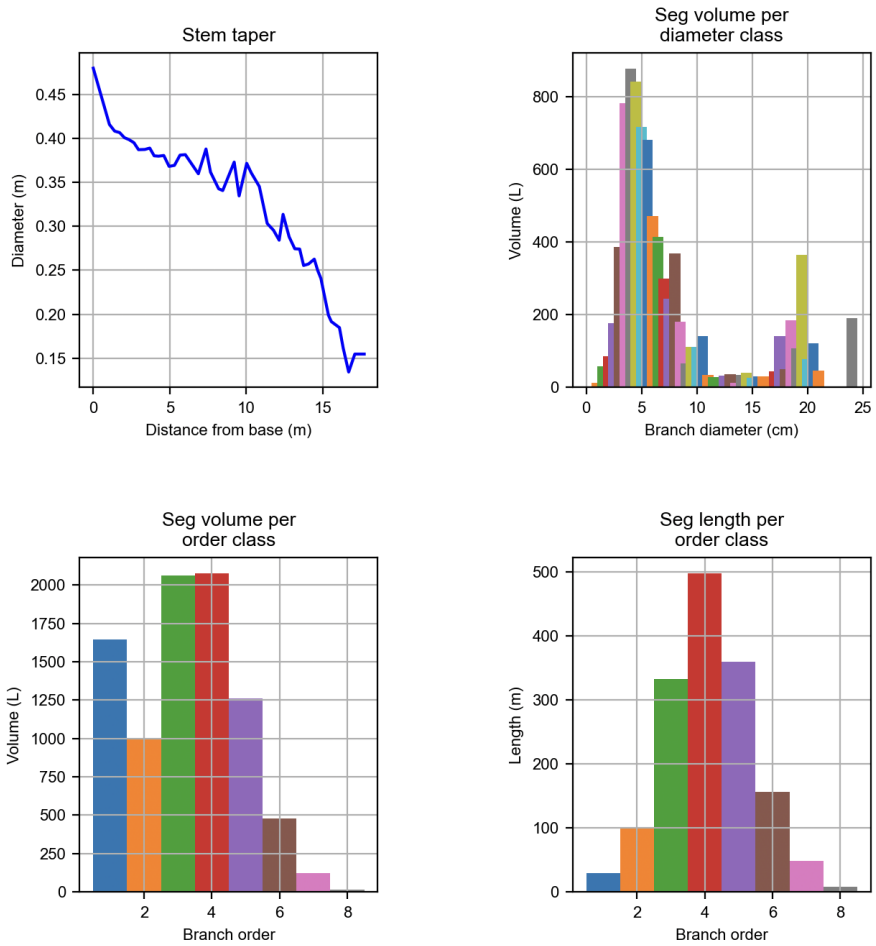

Figure 4: Example output data from sample pine. The output plots allow the user to derive summaries of various aspects of the input tree viewed within different categories including segment measurements by angle, direction and diameter class, as well as overall stem taper

Both treeqsm.py and treeqsm_batch.py may be run directly from the command line. This allows users to integrate the same functionality provided in the GUI into their own scripts with ease, whether those scripts are in python or not. Python users can use the package directly and get the full functionality by importing treeqsm.

Availability and Installation

The latest development version of PyTLidar as well as usage instructions are available at this GitHub repository. The package requires Python 3.8+ and a few key dependencies listed in the requirements. Installation instructions and example datasets are provided in the repository documentation. The latest release version is available on PyPi and can be installed using `pip install PyTLidar`.

Future Additions

While the initial release is focused on porting only TreeQSM, several future additions to PyTLidar are planned. The first planned enhancement is to provide a novel pipeline for

analyzing lidar scans of entire forest ecosystems to quantify vegetation structure at particular locations. This would allow users to load a series of lidar scan tiles and GPS observations of fauna and directly measure the environments, providing greater insights on components of habitat structural complexity.

Other planned enhancements include functions provided to users for processing lidar point clouds, including but not limited to various methods to perform ground filtering, tree segmentation and leaf/wood separation. The intended goal for this package is to provide a single source for any user processing terrestrial lidar to perform every step of their analysis.

Acknowledgements

We acknowledge contributions and guidance during the development of the package from Dori Peters, Amir Hossein Alikhah Mishamandani and other staff from the Human-Augmented Analytics Group to make this happen.

References

Atkins, D. H., Blackburn, R. C., Laughlin, D. C., Moore, M. M., & Sánchez Meador, A. J. (2025). Handheld lidar sensors can accurately measure aboveground biomass. *Ecosphere*, *16*(6), e70232. https://doi.org/10.1002/ecs2.70232

Bubnik V., T. J. (2025). 3D forest. In *GitHub repository*. Github. https://github.com/VUKOZ-OEL/3d-forest

Cannon, J., Zampieri, N. E., Whelan, A. W., Shearman, T. M., & Varner, J. M. (2024). Preprint: Terrestrial lidar scanning reduces subjectivity in measurements of tree fire injury. *SSRN Electronic Journal - Preprint*. https://doi.org/10.2139/ssrn.5056546

Donager, J. J., Sánchez Meador, A. J., & Blackburn, R. C. (2021). Adjudicating perspectives on forest structure: How do airborne, terrestrial, and mobile lidar-derived estimates compare? *Remote Sensing*, *13*(12). https://doi.org/10.3390/rs13122297

Fan G, D. Y., Nan L. (2020). AdQSM: A new method for estimating above-ground biomass from TLS point clouds. *Remote Sensing*, *12*(18), 3089. https://doi.org/10.3390/rs12183089

Hackenberg, J., Spiecker, H., Calders, K., Disney, M., & Raumonen, P. (2015). SimpleTree —an efficient open source tool to build tree models from TLS clouds. *Forests*, *6*(11), 4245–4294. https://doi.org/10.3390/f6114245

Hakala, T., Nevalainen, O., Kaasalainen, S., & Mäkipää, R. (2015). Technical note: Multispectral lidar time series of pine canopy chlorophyll content. *Biogeosciences*, *12*(5), 1629–1634. https://doi.org/10.5194/bg-12-1629-2015

Jan, H., Kim, C., Demol, M., Raumonen, P., Piboule, A., & Mathias, D. (2021). SimpleForest - a comprehensive tool for 3d reconstruction of trees from forest plot point clouds. *bioRxiv*. https://doi.org/10.1101/2021.07.29.454344

Lau, A., Bentley, L. P., Martius, C., Shenkin, A., Salgado-Labouriau, M., Jackson, T., Malhi, Y., Raumonen, P., Herault, B., Baraloto, C., Phillips, O. L., Gloor, E., Asner, G. P., & Enquist, B. J. (2018). Quantifying branch architecture of tropical trees using terrestrial LiDAR and 3D modelling. *Trees*, *32*(4), 1219–1231. https://doi.org/10.1007/s00468-018-1704-1

Loudermilk, E. L., Pokswinski, S., Hawley, C. M., Maxwell, A., Gallagher, M. R., Skowronski, N. S., Hudak, A. T., Hoffman, C., & Hiers, J. K. (2023). Terrestrial laser scan metrics predict surface vegetation biomass and consumption in a frequently burned southeastern u.s. ecosystem. *Fire*, *6*(4). https://doi.org/10.3390/fire6040151

Markku, Å., Raumonen, P., Kaasalainen, M., & Casella, E. (2015). Analysis of geometric

DRAFT

primitives in quantitative structure models of tree stems. *Remote Sensing*, *7*(4), 4581–4603. https://doi.org/10.3390/rs70404581

Martin-Ducup M., L. B. (2022). aRchi: A r package for the analysis of tree architecture. In *GitHub repository*. Github. https://github.com/umr-amap/aRchi

Metz, J., Seidel, D., Schall, P., Scheffer, D., Schulze, E.-D., & Ammer, C. (2013). Crown modeling by terrestrial laser scanning as an approach to assess the effect of aboveground intra- and interspecific competition on tree growth. *Forest Ecology and Management*, *310*, 275–288. https://doi.org/10.1016/j.foreco.2013.08.014

Raumonen, P., Åkerblom, M., Kaasalainen, M., & others. (2013). *TreeQSM: Quantitative structure models of trees from terrestrial laser scanning point clouds*. https://github.com/InverseTampere/TreeQSM.

Terryn, L., Calders, K., Disney, M., Origo, N., Malhi, Y., Newnham, G., Raumonen, P., Å kerblom, M., & Verbeeck, H. (2020). Tree species classification using structural features derived from terrestrial laser scanning. *ISPRS Journal of Photogrammetry and Remote Sensing*, *168*, 170–181. https://doi.org/10.1016/j.isprsjprs.2020.08.009

Zhu, Z., Kleinn, C., & Nölke, N. (2020). Assessing tree crown volume—a review. *Forestry: An International Journal of Forest Research*, *94*(1), 18–35. https://doi.org/10.1093/forestry/cpaa037

DRAFT